Temporal Difference Methods

Taxi-v3 Experiments

1 Introduction

Temporal Difference (TD) is kind of a middle ground between Monte Carlo (MC) methods and Dynamic Programming (DP).

Instead of waiting till the end of an episode like MC, TD updates values at every step. And unlike DP, it does not need a full model of the environment.

In simple words, TD learning means: *"learn from what you see now, and use your current guess of the future."*.

2 Environment: Taxi-v3

Taxi-v3 environment:

- Total states: 500
- Actions: 6 (move + pick/drop)
- Rewards:
  - +20 for correct drop
  - -1 per step
  - -10 for wrong pickup/drop
- Episode ends when task is done or after 200 steps

3 TD(0) Prediction

TD(0) is used to estimate how good each state is.

3.1 Update Rule

$$V(S_t) = V(S_t) + \alpha\big[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big] \tag{1}$$

The term inside brackets is called the **TD error**.

3.2 What does this mean?

- We compare what we expected vs what actually happened
- If the next state is better than expected $\rightarrow$ increase value
- If worse $\rightarrow$ decrease value

3.3 TD vs Monte Carlo

- TD updates every step
- MC updates only at the end
- TD is faster but slightly biased

3.4 Convergence

TD(0) works well if:

- All states are visited enough times
- Learning rate is chosen properly

3.5 Results

Observations:

- 10k episodes → values look rough
- 200k episodes → much smoother
- Most values are negative (because of -1 step penalty)

Also, $V(0)$ stayed 0, which is fine. That state is not clearly shown in the plotted slice. These results are consistent with Sutton & Barto (Section 6.1), where TD(0) is shown to learn efficiently through bootstrapping while updating estimates at every step. The gradual stabilization of values with more episodes matches the expected convergence behavior described in the textbook.

4 SARSA (On-Policy)

SARSA learns values using the actions it actually takes.

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big] \tag{2}$$

4.1 Key idea

- Uses current policy to learn
- Uses ϵ-greedy for exploration

4.2 GLIE Condition

SARSA works well when:

- We keep exploring (not stuck)
- Slowly become greedy over time

4.3 Simple intuition

SARSA learns a **safe strategy** because it considers the randomness in its actions.

This behavior aligns with Sutton & Barto (Section 6.4), where SARSA is described as an on-policy method that learns safer strategies due to incorporating the agent's exploration into updates. This explains why SARSA often produces smoother learning curves.

5 Q-Learning (Off-Policy)

Q-Learning learns using the **best possible action**, even if it wasn't taken.

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\big] \tag{3}$$

5.1 Key idea

- Learns optimal policy directly
- Faster but more aggressive

5.2 Cliff Walking idea

- SARSA → safe path
- Q-Learning → shortest but risky path

As explained in Sutton & Barto (Section 6.5), Q-Learning is an off-policy method that directly learns the optimal policy using the max operator. This leads to faster learning but can sometimes result in riskier behavior compared to SARSA.

Backup Diagrams (SARSA vs Q-Learning)

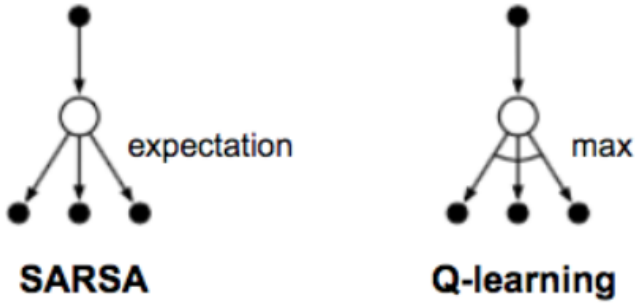


Figure 1: Backup diagrams for SARSA (expectation) and Q-Learning (max)

In SARSA, the update is based on the action actually taken in the next state. This is why it is shown as taking an expectation over possible actions, since the policy is ϵ-greedy and includes exploration. As explained in Sutton & Barto (Section 6.4), this makes SARSA an on-policy method.

In contrast, Q-Learning always updates using the maximum possible action value at the next state, regardless of what action was actually taken. This is shown by the "max" operation in the diagram. According to Sutton & Barto (Section 6.5), this makes Q-Learning an off-policy method that learns the optimal policy directly.

This difference explains why SARSA tends to learn safer policies, while Q-Learning can learn faster but more aggressive strategies.

This difference is also clearly seen in the Cliff Walking example discussed in Sutton & Barto.

6 Experiment 2: SARSA vs Q-Learning

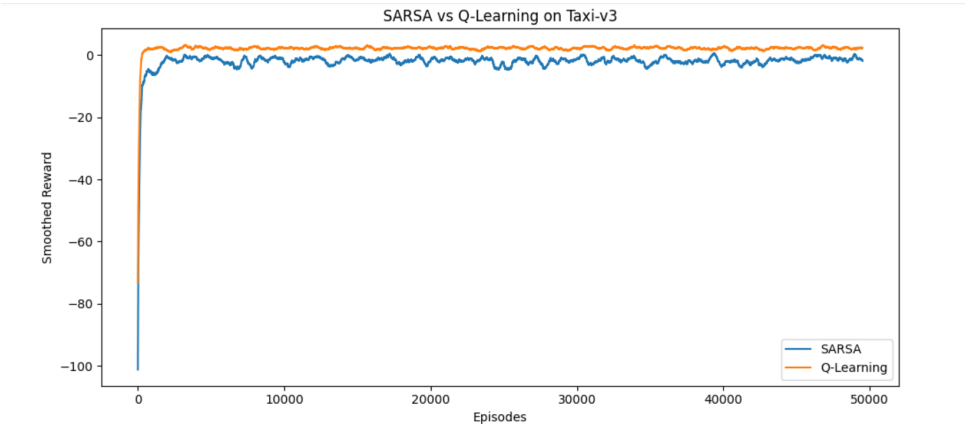


Figure 2: Reward comparison

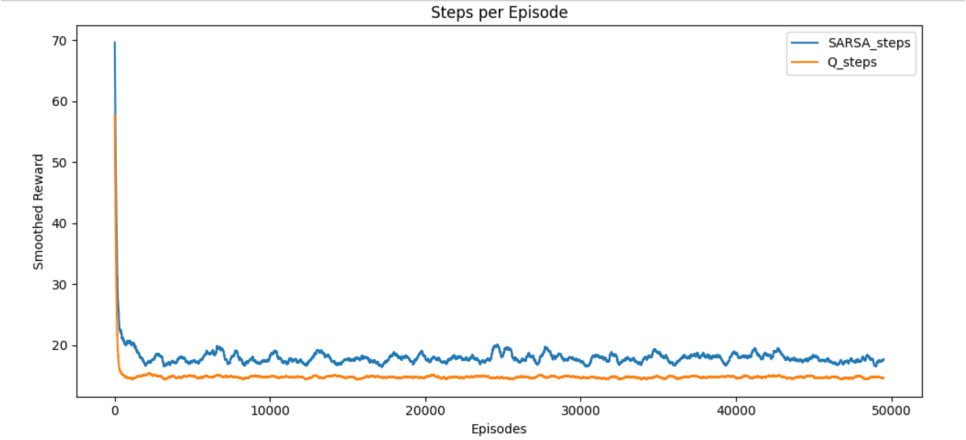


Figure 3: Steps per episode

What we see:

- Q-Learning learns faster
- SARSA is smoother
- Q-Learning gets better rewards

6.1 Policies

Q-Learning gives more direct paths, SARSA is slightly careful.

7 n-Step SARSA

$$G_t^{(n)} = R_{t+1} + \cdots + \gamma^n Q(S_{t+n}, A_{t+n}) \tag{4}$$

7.1 Idea

- Small $n \rightarrow$ fast but biased
- Large $n \rightarrow$ accurate but noisy

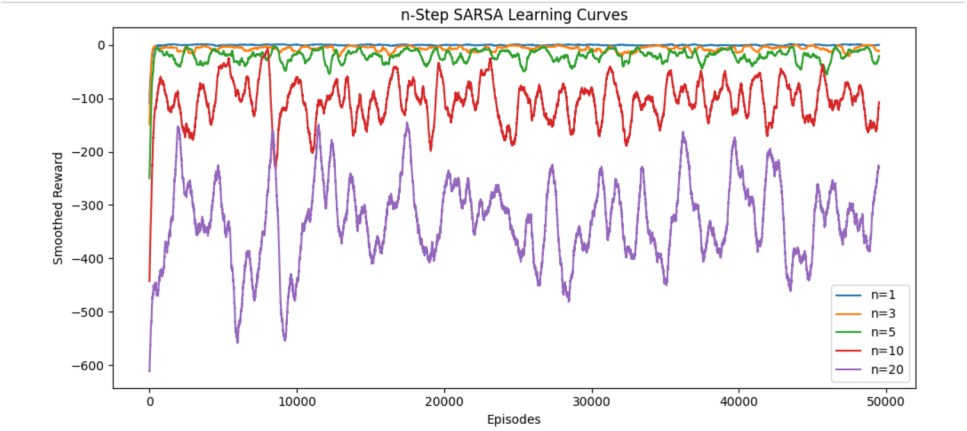


Figure 4: n-step performance

Observation:

- n = 3 or 5 works best
- Very large n becomes unstable

This matches the discussion in Sutton & Barto (Chapter 7), where n-step methods are shown to balance bias and variance. Small values of n learn quickly but are biased, while larger values reduce bias but increase variance.

8 TD(λ)

TD(λ) is a method that combines all n-step returns into a single update. Instead of choosing a fixed n, it takes a weighted average of all possible n-step returns.

$$G^{\lambda} = (1-\lambda)\sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)} \tag{5}$$

Eligibility Traces

A key idea in TD(λ) is the use of eligibility traces. Each state keeps a trace value $e_t(s)$ which shows how recently and how frequently it has been visited.

At each step, the TD error is computed as:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \tag{6}$$

Then, all states are updated using:

$$V(s) \leftarrow V(s) + \alpha \delta_t e_t(s) \tag{7}$$

Intuition

The idea is simple: instead of updating only the current state, TD(λ) spreads the update backward to previously visited states. States that were visited more recently get larger updates.

- $\lambda = 0$ behaves like TD(0) (only one-step updates)
- $\lambda = 1$ behaves like Monte Carlo (full return)
- Intermediate values balance bias and variance

As discussed in Sutton & Barto (Chapter 12), TD(λ) provides a smooth way to trade off between fast but biased learning (TD) and slow but accurate learning (MC). It is also more efficient than explicitly computing all n-step returns.

9 Hyperparameter Study

9.1 Learning Rate (α)

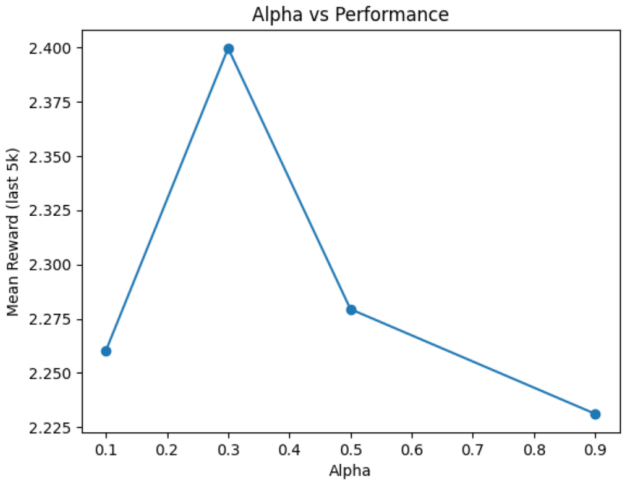


Figure 5: Alpha vs performance

Best at $\alpha = 0.3$

9.2 Discount (γ)

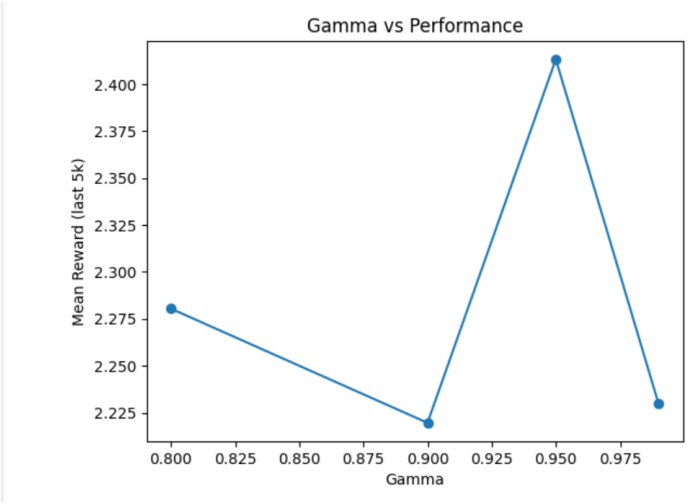


Figure 6: Gamma vs performance

Best at $\gamma = 0.95$

The sensitivity to hyperparameters observed here is also highlighted in Sutton & Barto, where learning rate, discount factor, and exploration all significantly affect convergence speed and stability.

9.3 Exploration (ϵ)

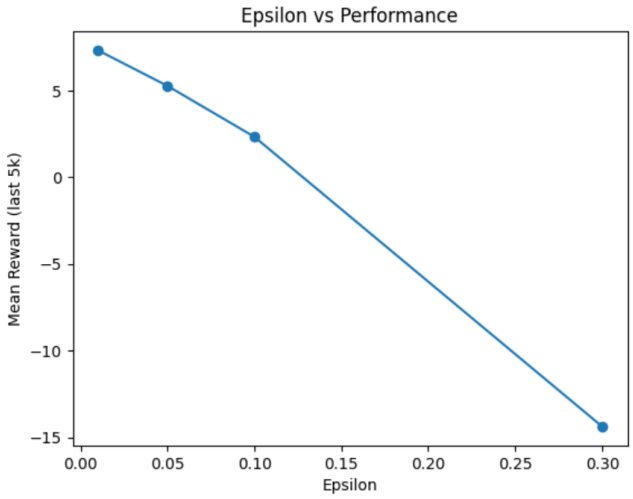


Figure 7: Epsilon vs performance

Lower ϵ works better here.

10 Discussion

Some important takeaways:

- TD learning is fast because of bootstrapping

- Q-Learning is faster but riskier
- SARSA is safer but slower
- n-step methods show clear tradeoff
- Hyperparameters matter a lot

One interesting thing was how even small changes in ϵ affected results a lot.

11 Conclusion

From these experiments:

- TD(0) learns values gradually
- Q-Learning performs best overall
- n-step methods balance learning well
- Good parameter tuning is very important

Overall, the results match what we expect from Sutton & Barto, showing that TD methods work well in practice.